Manual de uso

AGR — Práctica 2: Virtualización con Vagrant y QEMU-KVM

Pablo Elvira Calatayud

Índice

- 1. Contenido del repositorio
- 2. Parte 1: servidor web con Vagrant
  - 2.1. Instalación (una sola vez)
  - 2.2. Uso
  - 2.3. Comprobaciones
  - 2.4. Problemas frecuentes
- 3. Parte 2: red del laboratorio con KVM
  - 3.1. Instalación (una sola vez)
  - 3.2. Órdenes del script
  - 3.3. Primer despliegue
  - 3.4. Pruebas
  - 3.5. Modificar la topología
  - 3.6. Medir el consumo
  - 3.7. Problemas frecuentes
- 4. Regenerar la documentación

1. Contenido del repositorio

```
AGR/
├── parte1-vagrant/Vagrantfile     Parte 1: VM con el servidor web (VirtualBox)
├── parte2-kvm/                    Parte 2: red del laboratorio 3103 (KVM)
│   ├── agr_kvm.py                 script de despliegue
│   ├── topologia.json             plan de numeración (editable)
│   ├── plantilla_vm.xml           XML base de Moodle
│   └── guest/                     ficheros que se instalan en las VMs
└── doc/
    ├── entregables/               PDF finales (+ fuentes en src/)
    ├── development/               enunciado, plano, medidas, informes
    └── build_pdf.sh               regenera los PDF
```

Requisitos del anfitrión: Linux x86-64 Debian/Ubuntu (probado en Ubuntu 24.04), CPU con virtualización (VT-x/AMD-V) activada en la BIOS, ≥ 8 GB de RAM y ≥ 10 GB libres de disco, y conexión a Internet la primera vez.

```
        git clone https://github.com/PabloElviraCalatayud/AGR.git
        cd AGR
```

> **VirtualBox (parte 1) y KVM (parte 2) no pueden ejecutarse a la vez.** Las herramientas lo gestionan solas: `agr_kvm.py` descarga VirtualBox y carga KVM antes de arrancar las VMs y descarga KVM al `parar`/`destruir`; el `Vagrantfile` hace lo contrario antes de `vagrant up` (pide la contraseña de `sudo`). Sólo hay que apagar las VMs de una parte antes de usar la otra: `vagrant halt` o `sudo ./agr_kvm.py parar`.

2. Parte 1: servidor web con Vagrant

2.1. Instalación (una sola vez)

```
        # VirtualBox 7.2 (repositorio de Oracle)
        sudo apt install -y wget gpg build-essential dkms linux-headers-$(uname -r)
        wget -O- https://www.virtualbox.org/download/oracle_vbox_2016.asc \
          | sudo gpg --dearmor --yes -o /usr/share/keyrings/oracle-virtualbox-2016.gpg
        echo "deb [arch=amd64 signed-by=/usr/share/keyrings/oracle-virtualbox-2016.gpg]
https://download.virtualbox.org/virtualbox/debian $(lsb_release -cs) contrib" \
          | sudo tee /etc/apt/sources.list.d/virtualbox.list
        sudo apt update && sudo apt install -y virtualbox-7.2
        sudo usermod -aG vboxusers $USER          # cerrar sesión y volver a entrar

        # Vagrant (repositorio de HashiCorp)
        wget -O- https://apt.releases.hashicorp.com/gpg \
          | sudo gpg --dearmor --yes -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
        echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com
$(lsb_release -cs) main" \
          | sudo tee /etc/apt/sources.list.d/hashicorp.list
        sudo apt update && sudo apt install -y vagrant

        VBoxManage --version     # sin WARNING
        vagrant --version
```

2.2. Uso

Acción	Orden (desde `parte1-vagrant/`)
Crear y arrancar la VM (1ª vez: descarga e instala todo)	`vagrant up`
Entrar (muestra la configuración del servidor)	`vagrant ssh`
Estado	`vagrant status`
Volver a aplicar el aprovisionamiento	`vagrant provision`
Reiniciar	`vagrant reload`
Apagar	`vagrant halt`
Borrar la VM	`vagrant destroy -f`

Opciones (variables de entorno antes de `vagrant up`):

Variable	Efecto	Ejemplo
`VM_MEMORY`	RAM en MB (por defecto 1024)	`VM_MEMORY=512 vagrant up`
`VM_CPUS`	número de CPUs (por defecto 1)	`VM_CPUS=2 vagrant up`
`VM_GUI`	abre la ventana de VirtualBox (login `vagrant`/`vagrant`)	`VM_GUI=true vagrant up`
`BRIDGE_IF`	añade una interfaz pública en puente con esa NIC física	`BRIDGE_IF=enp3s0 vagrant up`

2.3. Comprobaciones

```
        curl http://localhost:8080                          # anfitrión -> VM (port
forwarding)
        curl http://192.168.56.10:3000                      # red privada
        vagrant ssh -c "curl -s localhost:3000"             # dentro de la VM
        curl "http://localhost:8080/record?
id_nodo=n1&temperatura=24.5&humedad=60&co2=300&volatiles=100"
        # Navegador: http://localhost:8080  y  http://localhost:8080/logs
```

Dentro de la VM: `systemctl status ssr-server`, `journalctl -u ssr-server -f`, `sudo ufw status`.

2.4. Problemas frecuentes

Síntoma	Solución
`No usable default provider` / `vboxdrv kernel module is not loaded`	`sudo /sbin/vboxconfig`; si falla, ver filas siguientes
`modprobe vboxdrv failed` con Secure Boot	Desactivar Secure Boot en la BIOS (o registrar la clave MOK) y repetir `sudo /sbin/vboxconfig`
VirtualBox no compila para un kernel nuevo	Instalar la última versión (`virtualbox-7.2`)
`VERR_VMX_IN_VMX_ROOT_MODE` / `VERR_SVM_IN_USE`	Hay VMs de KVM encendidas: `sudo ./agr_kvm.py parar` (en `parte2-kvm/`)
`A Vagrant environment ... is required`	Ejecutar desde `parte1-vagrant/`
`Vagrant cannot forward the specified ports` (8080 ocupado)	Liberar el puerto 8080 en el anfitrión o cambiar `PUERTO_HOST` en el `Vagrantfile`

3. Parte 2: red del laboratorio con KVM

3.1. Instalación (una sola vez)

```
        cd parte2-kvm
        sudo ./agr_kvm.py instalar
```

Instala QEMU/KVM, libvirt, libguestfs, `lxml` y las herramientas de red, y activa `libvirtd`.

Imagen de las VMs. Se usa la imagen de disco de Moodle (Ubuntu 20.04, `root`/`agr`), como recomienda el enunciado. Cópiela en `parte2-kvm/imagen/agr-vm-base.qcow2`; si no está, `crear` la descarga del Release `imagen-moodle` del repositorio. No se versiona en git por su tamaño (~2 GB).

3.2. Órdenes del script

Todas desde `parte2-kvm/` y con `sudo` (salvo `xml`). `-v` muestra trazas.

Orden	Qué hace
`sudo ./agr_kvm.py instalar`	Instala las dependencias del anfitrión (una vez)
`sudo ./agr_kvm.py crear`	Crea la imagen base (sólo la 1ª vez), los bridges, las 9 VMs y las arranca. Si ya existían, las recrea
`sudo ./agr_kvm.py crear rc pc1`	Recrea sólo esas máquinas
`sudo ./agr_kvm.py estado`	Estado, rol, RAM e IPs de cada VM
`sudo ./agr_kvm.py probar [--informe f.txt] [--detalle]`	Plan de pruebas de conectividad (ver §3.4)
`sudo ./agr_kvm.py consola ra pc1`	Abre un `xterm` con la consola serie de cada VM
`sudo ./agr_kvm.py consumo [--informe f.txt]`	RAM real (RSS), CPU y disco de cada VM y total
`sudo ./agr_kvm.py parar [vms]`	Apaga ordenadamente (y libera KVM para VirtualBox)
`sudo ./agr_kvm.py arrancar [vms]`	Arranca (y recrea bridges y rutas del anfitrión, que se pierden al reiniciar el PC)
`sudo ./agr_kvm.py destruir [vms]`	Borra VMs y discos; sin argumentos, también bridges y rutas
`sudo ./agr_kvm.py destruir --todo`	Además borra la imagen base construida (no la de `imagen/`)
`./agr_kvm.py xml --salida dir`	Sólo genera XML, `frr.conf` y `node.conf` para revisarlos

Opciones de `crear`:

Opción	Uso
`--imagen-base RUTA.qcow2`	Partir de otra imagen (por defecto, la de Moodle en `imagen/`); `ubuntu` usa Ubuntu 24.04 cloud
`--reconstruir-base`	Rehacer la imagen base (p. ej. tras cambiar `guest/`)
`--proxy http://host:puerto`	Proxy para descargar paquetes (redes universitarias)
`--espera N`	Segundos máximos de espera al arranque (300)
`--dir RUTA` (global)	Directorio de discos y XML (`/var/lib/libvirt/images/agr`)

3.3. Primer despliegue

```
cd parte2-kvm
sudo ./agr_kvm.py -v crear     # 1ª vez ~5-10 min (construye la imagen base)
sudo ./agr_kvm.py estado       # las 9 deben estar "running"
```

VM	Rol	IP(s)	Acceso
`agr-ra`	router	10.100.0.1, 10.200.0.1, 10.1.0.1	`ssh root@10.200.0.1`
`agr-rb`	router	10.1.0.2, 10.2.0.1, 10.3.0.1	`ssh root@10.1.0.2`
`agr-rc`	router	10.2.0.2, 10.0.1.1, 10.0.2.1	`ssh root@10.2.0.2`
`agr-rd`	router	10.3.0.2, 10.0.3.1, 10.0.4.1	`ssh root@10.3.0.2`
`agr-srv`	servidor web :3000	10.100.0.11	`ssh root@10.100.0.11`
`agr-pc1..4`	PC	10.0.1.10 … 10.0.4.10	`ssh root@10.0.1.10`
anfitrión	PC Profesor	10.200.0.2 (bridge `agr-prof`)	—

Usuario `root`, contraseña `agr` en todas (consola). Por SSH se entra con la clave del usuario que ejecutó `sudo`, si tenía una en `~/.ssh/`.

3.4. Pruebas

Automáticas:

```
        sudo ./agr_kvm.py probar --informe ../doc/development/parte2/pruebas.txt
```

Manuales:

```
        curl http://10.100.0.11:3000/               # anfitrión -> servidor web (o en el
navegador)
        traceroute -n 10.0.3.10                     # anfitrión -> PC3: RA, RB, RD, PC3
        sudo ./agr_kvm.py consola pc1               # en PC1 (root/agr):
            curl http://10.100.0.11:3000/
            traceroute -n 10.100.0.11               #   RC -> RB -> RA -> SRV
            ping -c3 10.0.4.10                      #   PC1 -> PC4
        sudo ./agr_kvm.py consola rb                # en RB:
            vtysh -c 'show ip route'                #   tabla de rutas (FRR)
            vtysh -c 'show running-config'
            vtysh -c 'show interface brief'
```

Salir de la consola: `Ctrl+]`. Al iniciar sesión, cada VM muestra su rol, interfaces, rutas y el estado de FRR o del servidor web.

3.5. Modificar la topología

Toda la red está en `topologia.json`: redes (bridge y CIDR), máquinas (rol, RAM, interfaces con IP, rutas). Tras editarlo:

```
        ./agr_kvm.py xml --salida /tmp/revision     # revisar lo que se va a generar
        sudo ./agr_kvm.py destruir && sudo ./agr_kvm.py crear
```

El script comprueba antes de desplegar que cada IP pertenece a la red a la que se conecta.

3.6. Medir el consumo

```
sudo ./agr_kvm.py consumo --informe ../doc/development/parte2/consumo.txt
```

Muestra, por VM, la RAM asignada, la RAM real que ocupa en el anfitrión (RSS del proceso QEMU), el disco diferencial y el tiempo de CPU, y los totales frente a la RAM del anfitrión. Repitiéndolo durante `probar` se observa el crecimiento de los discos.

3.7. Problemas frecuentes

Síntoma	Causa / solución
Aviso `/dev/kvm no existe`	Virtualización (VT-x/AMD-V) desactivada en la BIOS; el script ya intenta cargar el módulo
`No se encuentra la imagen de Moodle`	Copiarla en `parte2-kvm/imagen/agr-vm-base.qcow2`
VMs no arrancan: `Permission denied` en el disco	El directorio no es accesible para `libvirt-qemu`: usar el `--dir` por defecto
`virt-customize` no descarga paquetes	Falta `isc-dhcp-client` en el anfitrión o hace falta `--proxy`
`supermin exited with error`	libguestfs necesita un kernel en `/boot`; ejecutar con `-v` para ver el detalle
Un router no tiene rutas	`sudo ./agr_kvm.py crear <router>` lo recrea; la imagen ya reaplica la configuración de FRR al arrancar
Tras reiniciar el PC no hay conectividad con las VMs	Los bridges no son persistentes: `sudo ./agr_kvm.py arrancar`
Sin conectividad anfitrión ↔ lab con Docker instalado	El script desactiva `bridge-nf-call-iptables`; revisar `sudo iptables -L FORWARD`
Aviso `VirtualBox está en uso`	Apagar la VM de la parte 1: `vagrant halt`

4. Regenerar la documentación

```
sudo apt install pandoc chromium-browser     # o google-chrome
doc/build_pdf.sh                             # doc/entregables/src/*.md ->
doc/entregables/*.pdf
```